

Satellite Tracker OSINT Tool - Implementation Specification

Overview

Build a web-based satellite intelligence tracker for The Rudd Report OSINT toolkit. This tool allows users to track satellites in real-time, predict passes over locations, and analyze orbital coverage - critical for geospatial intelligence and surveillance awareness.

Technical Stack

- **Framework:** Next.js 15 (App Router) + TypeScript
- **Location:** `app/tools/satellite-tracker/page.tsx`
- **Orbit Propagation:** satellite.js library
- **TLE Data Source:** CelesTrak (free, public API)
- **Visualization:**
 - 3D globe: Three.js with Earth texture
 - 2D ground track: Leaflet or Canvas-based map
- **Styling:** Tailwind CSS (consistent with existing tools)

Core Features

1. Satellite Search & Selection

- Search by satellite name or NORAD catalog ID
- Popular presets:
 - International Space Station (ISS)
 - Starlink constellation
 - GPS satellites
 - Iridium satellites
 - Military/reconnaissance satellites (USA-series)
 - Weather satellites (NOAA, GOES)
 - Communication satellites

2. Real-time Tracking

- Current position (lat/lon/altitude)
- Current velocity

- Orbital period
- Inclination
- Apogee/Perigee
- Time to next pass over user location
- Azimuth and elevation

3. Pass Predictions

- User inputs observer location (lat/lon or city name)
- Calculate next 10 passes
- Show for each pass:
 - AOS (Acquisition of Signal) time
 - LOS (Loss of Signal) time
 - Maximum elevation angle
 - Duration
 - Direction (ascending/descending)
 - Visibility (daylight/night/twilight)

4. 3D Visualization

- Rotating Earth globe with satellite position marker
- Orbital path visualization
- Ground track (subsatellite point trail)
- Real-time updates (1-second intervals)
- Camera controls (zoom, rotate, pan)

5. Coverage Analysis

- "What satellites can see this location now?"
- Input target coordinates
- Show all satellites with line-of-sight
- Minimum elevation filter (e.g., $>10^\circ$)
- Time window for coverage

Data Sources

TLE Data (Two-Line Elements)

Use Celestrak API - no authentication required:

```
https://celestrak.org/NORAD/elements/gp.php?GROUP=active&FORMAT=tle
https://celestrak.org/NORAD/elements/gp.php?GROUP=stations&FORMAT=tle
https://celestrak.org/NORAD/elements/gp.php?GROUP=weather&FORMAT=tle
```

Satellite Categories to Support

- Space Stations (ISS, Tiangong)
- Weather satellites
- Navigation (GPS, GLONASS, Galileo, BeiDou)
- Communication satellites
- Earth observation
- Military/intelligence satellites
- Amateur radio satellites

UI/UX Design

Layout

Satellite Tracker

[Search: Enter satellite name or NORAD ID]

Quick Select: [ISS] [Starlink] [GPS] [Spy]

3D Globe Visualization

Current Position:

Lat: 34.52° N

Lon: 120.34° W

Alt: 408 km

Velocity: 7.66 km/s

Orbital Parameters:

Period: 92.5 min

Inclination: 51.6°

Observer Location:

[Lat input] [Lon input] or [City search]

Next Pass Predictions:

```
| | AOS: Mar 21, 19:45 UTC | | |
| | LOS: Mar 21, 19:52 UTC | |
| | Max Elevation: 45° | Duration: 7m 12s | |
| | Direction: SW→NE | Visible: Yes | |
| |-----| |
| [Show more passes...] |
```

Color Scheme (Match existing tools)

- Background: Dark slate
- Accent: Blue/cyan (intel theme)
- Text: Light gray on dark
- Satellite marker: Bright cyan/yellow
- Orbital path: Translucent blue
- Ground track: Yellow/orange

Implementation Steps

1. Install Dependencies

```
bash

npm install satellite.js three @react-three/fiber @react-three/drei
npm install @types/three
```

2. Create API Routes

- `app/api/satellite/tle/route.ts` - Fetch TLE data from Celestrak
- `app/api/satellite/search/route.ts` - Search satellites by name

3. Create Main Component

```
app/tools/satellite-tracker/page.tsx
```

- Use `'use client'` directive
- State management for selected satellite, observer location, TLE data
- Real-time updates using `setInterval`

4. Create Subcomponents

- `SatelliteSearch.tsx` - Search and selection UI
- `SatelliteInfo.tsx` - Display current position and orbital parameters
- `PassPredictor.tsx` - Calculate and display pass predictions
- `Globe3D.tsx` - Three.js 3D Earth with satellite
- `GroundTrack.tsx` - 2D map showing ground track

5. Utilities

`lib/satellite-utils.ts`:

- `propagateSatellite()` - Calculate position from TLE
- `calculatePasses()` - Predict satellite passes
- `getVisibility()` - Determine if pass is visible
- `getSatellitePosition()` - Get lat/lon/alt from TLE

satellite.js Core Functions

typescript

```
import * as satellite from 'satellite.js';

// Parse TLE
const satrec = satellite.twoline2satrec(tleLine1, tleLine2);

// Get position at time
const positionAndVelocity = satellite.propagate(satrec, new Date());
const positionEci = positionAndVelocity.position;

// Convert to lat/lon
const gmst = satellite.gstime(new Date());
const positionGd = satellite.eciToGeodetic(positionEci, gmst);

const latitude = satellite.degreesLat(positionGd.latitude);
const longitude = satellite.degreesLong(positionGd.longitude);
const altitude = positionGd.height; // km
```

OSINT Use Cases to Highlight

Include educational text explaining:

1. **Surveillance awareness:** Know when spy satellites can see your location
2. **Communication windows:** When satellites are in view for data uplink/downlink
3. **Geospatial intelligence:** Track satellite coverage over areas of interest
4. **Open-source intelligence:** All data is publicly available via CelesTrak
5. **Space situational awareness:** Understand orbital infrastructure

Performance Considerations

- Cache TLE data (update every 24 hours)
- Throttle 3D rendering updates to 1 FPS
- Use Web Workers for intensive calculations
- Lazy load satellite database
- Limit concurrent satellite tracking to 5 satellites

Testing

- Test with ISS (NORAD ID 25544)
- Verify pass predictions against known ISS passes
- Test with various observer locations
- Verify orbital calculations match reference data

Navigation Integration

Add to main OSINT tools navigation:

```
typescript

// In app/tools/page.tsx or navigation component
{
  name: 'Satellite Tracker',
  href: '/tools/satellite-tracker',
  description: 'Track satellites in real-time and predict passes',
  category: 'Geospatial Intelligence'
}
```

File Structure

```
app/
  tools/
```

```
satellite-tracker/  
  page.tsx          # Main tool page  
  components/  
    SatelliteSearch.tsx  
    SatelliteInfo.tsx  
    PassPredictor.tsx  
    Globe3D.tsx  
    GroundTrack.tsx  
api/  
  satellite/  
    tle/  
      route.ts      # TLE data fetching  
    search/  
      route.ts      # Satellite search  
lib/  
  satellite-utils.ts # Orbit calculations
```

Build Requirements

1. Must work in Next.js 15 App Router
2. Must be client-side rendered (`'use client'`)
3. Must handle async TLE data loading
4. Must be responsive (mobile-friendly)
5. Must match existing tool styling
6. Must include error handling for API failures

Additional Notes

- TLE data updates daily on CelesTrak
- Use `Date.now()` for real-time propagation
- Consider adding satellite categories filter
- May add "Export pass schedule" feature later
- Future: Add ISS crew information via Open Notify API

Implementation Command for Claude Code

To implement this in VS Code:

1. Save this file as `satellite-tracker-spec.md` in your ruddreport root
2. Open Claude Code in VS Code terminal
3. Run: Read `satellite-tracker-spec.md` and implement the satellite tracker tool following these specifications exactly. Use the existing `ruddreport.net` code style and structure.